**UNIVERSIDAD MANUELA BELTRÁN**

# ARQUITECTURA INICIAL Y ESTRUCTURA DEL PRODUCTO

PlanifiCPU · Sistemas Operativos · Grupo G11 · 2026

Equipo: David Zambrano, Daniel Pérez, Axel Salgado

## 1. Componentes principales y módulos funcionales

| Módulo | Archivo(s) | Responsabilidad |
|---|---|---|
| Interfaz gráfica | gui.py | Formularios, tablas, pestañas de resultados y diálogos de archivo (Tkinter + Matplotlib). |
| Gestor de procesos | modelos.py, csv_io.py | Crea y valida los procesos; carga escenarios desde CSV. |
| Motor de planificación | algoritmos.py | Un algoritmo por función con una interfaz común: simular(procesos, algoritmo, quantum, expropiativo). |
| Métricas y reportes | metricas.py, csv_io.py | Cálculo de tiempos, comparación y exportación a CSV/PNG. |
| Persistencia | bd.py, bd/schema.sql | Guarda escenarios, simulaciones, resultados y tramos del Gantt en SQLite. |

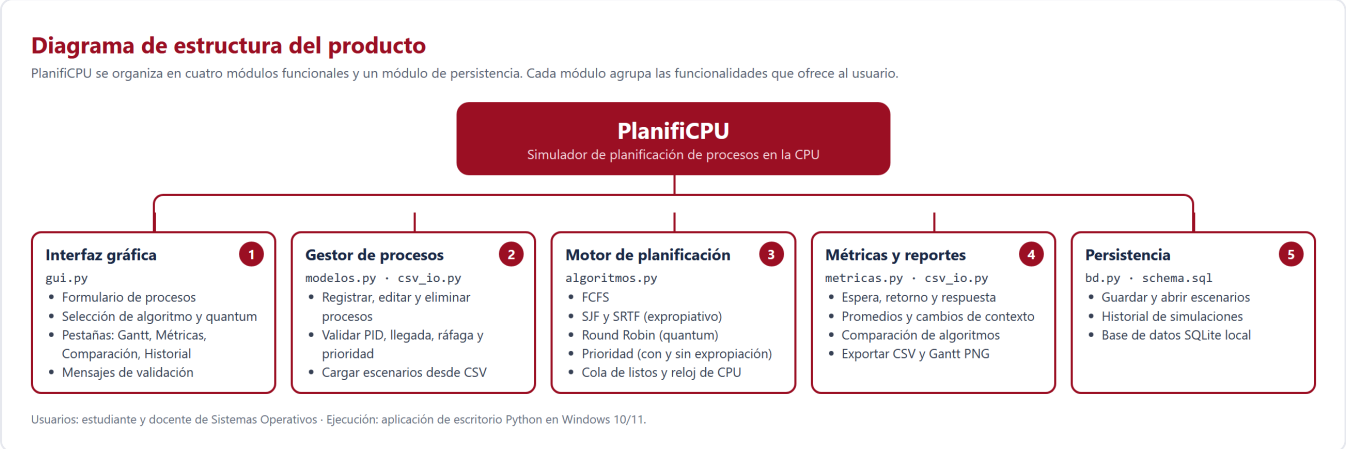



*Figura 1. Diagrama de estructura del producto.*

## 2. Arquitectura inicial de la solución

La aplicación sigue una arquitectura en tres capas dentro de un mismo programa de escritorio. La capa de presentación (Tkinter y Matplotlib) recoge los datos y muestra resultados, pero no calcula; delega en la capa de lógica, que valida los procesos, ejecuta el algoritmo y calcula las métricas. Solo la capa de datos lee y escribe archivos CSV/PNG y la base de datos SQLite. Esta separación permite probar el motor sin la interfaz y cambiar la interfaz sin tocar los algoritmos.




*Figura 2. Diagrama de arquitectura inicial en tres capas.*

### Flujo de una simulación

1. El usuario registra los procesos (o carga un CSV) y elige el algoritmo, el quantum y el modo expropiativo.
2. La interfaz llama a `simular()`; el motor valida los datos con `validar_procesos()`.
3. El motor simula el reloj de CPU y la cola de listos, y construye la lista de tramos del Gantt.
4. `calcular_metricas()` obtiene inicio, fin, espera, retorno y respuesta de cada proceso.
5. La interfaz dibuja el Gantt, el registro de ejecución y la tabla de métricas, y `bd.py` guarda la simulación.

## 3. Organización de archivos y directorios

### Organización de archivos y directorios

Estructura del repositorio PlanifiCPU_SistemasOperativos_G11 (basada en la estructura sugerida por la guía).

```
PlanifiCPU_SistemasOperativos_G11/
├── README.md
├── LICENSE
├── requirements.txt
├── documentacion/
│   ├── descripcion_proyecto.pdf
│   ├── requerimientos.pdf
│   ├── arquitectura_inicial.pdf
│   ├── tecnologias_y_referencias.pdf
│   └── diagramas/
├── src/
│   └── codigo_fuente/
│       ├── main.py
│       ├── planificpu/
│       │   ├── gui.py
│       │   ├── algoritmos.py
│       │   ├── metricas.py
│       │   ├── modelos.py
│       │   ├── csv_io.py
│       │   └── bd.py
│       └── bd/
│           └── schema.sql
├── recursos/
│   ├── imagenes/
│   └── datos_ejemplo/
└── pruebas/
    └── test_algoritmos.py
```

**documentacion/**
Documentos PDF del proyecto y diagramas (PNG y su fuente HTML).

**src/codigo_fuente/**
Aplicación: punto de entrada `main.py`, paquete `planificpu` con los módulos y el esquema de la base de datos.

**recursos/**
Capturas de la interfaz y escenarios CSV de ejemplo.

**pruebas/**
Pruebas automáticas del motor (unittest).

**Raíz**
README (presentación), licencia MIT y dependencias.

*Figura 3. Estructura del repositorio.*

## 4. Base de datos y modelo relacional

Motor SQLite 3 en el archivo `src/codigo_fuente/bd/planificpu.db`, que se crea automáticamente al abrir la aplicación a partir de `bd/schema.sql`. Tiene llaves foráneas activas, restricciones CHECK y borrado en cascada. El modelo está en tercera forma normal; los promedios de cada simulación se guardan a propósito para consultar el historial sin recalcular.

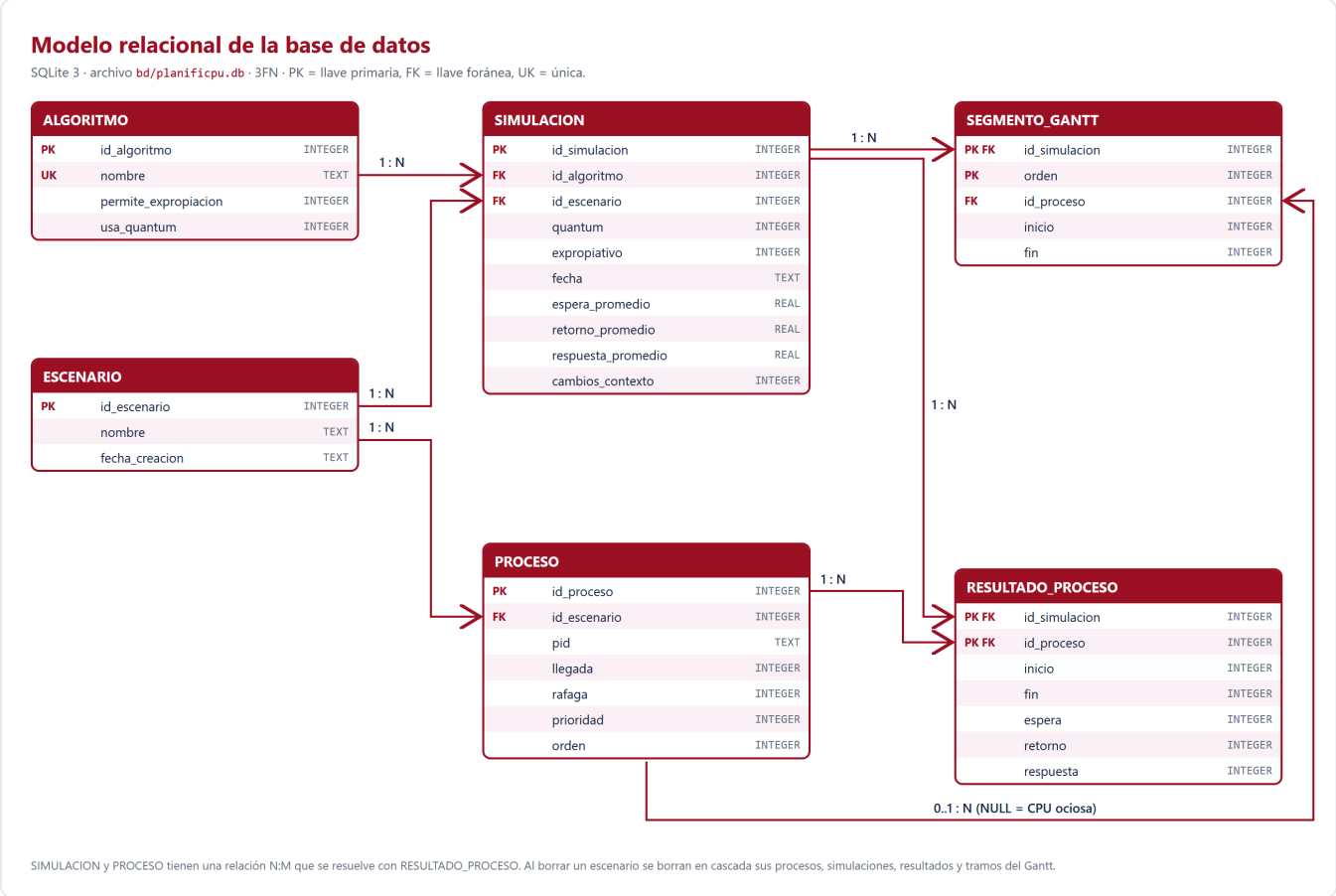



*Figura 4. Modelo relacional de la base de datos.*

## 5. Sistema operativo y entorno de ejecución previsto

| Elemento | Entorno previsto |
|---|---|
| Sistema operativo | Windows 10 u 11 de 64 bits (entorno previsto) |
| Intérprete | Python 3.12 o superior, con Tkinter y sqlite3 (incluidos en el instalador oficial) |
| Librerías | Matplotlib 3.8 o superior (requirements.txt) |
| Desarrollo | Visual Studio Code, Git 2.x y cuenta de GitHub |
| Hardware mínimo | Procesador: Doble núcleo de 1,6 GHz o superior; Memoria RAM: 4 GB (recomendado 8 GB); Almacenamiento: 500 MB libres (Python, Matplotlib y la aplicación); Pantalla: Resolución mínima de 1366 × 768; Periféricos: Teclado y mouse |

> No se requieren herramientas de virtualización: la aplicación corre directamente sobre Windows y no depende de servicios externos. La base de datos es un archivo local y no necesita servidor.